

# Space Shooter - Build Instructions

## Project Overview

A simple arcade-style **space shooter** game built with **Unity** and **C#**.

All gameplay elements (sprites, UI, game logic) are created **programmatically at runtime** — you only need to open the project in Unity, set up scenes, and build.

## Prerequisites

| Requirement   | Details                                                      |
|---------------|--------------------------------------------------------------|
| Unity Editor  | <b>2021.3 LTS</b> or newer (recommended: <b>2022.3 LTS</b> ) |
| Platform      | Windows 10/11 (for building a Windows standalone)            |
| Unity Modules | Windows Build Support (IL2CPP or Mono)                       |

💡 Download Unity Hub from <https://unity.com/download> then install the recommended LTS version. During installation, make sure to check **“Windows Build Support”**.

## Step-by-Step Setup

### Step 1 - Open the Project in Unity

1. Launch **Unity Hub**
2. Click **“Open”** → navigate to this project folder ( `space_shooter_game/` )
3. Select the folder and click **“Open”**
4. Unity will import assets and compile scripts (first time may take 1-2 minutes)

### Step 2 - Set Up Tags and Layers

The game requires specific tags. Unity may not auto-import them from the settings files. Verify/create these tags manually if needed:

**Tags** (Edit → Project Settings → Tags and Layers → Tags):

- Player
- PlayerBullet
- EnemyBullet
- Enemy
- PowerUp

**Layers** (same panel → Layers):

- Layer 8: PlayerBullet
- Layer 9: EnemyBullet
- Layer 10: Player
- Layer 11: Enemy
- Layer 12: PowerUp

## Step 3 - Set Up Scenes

1. In the **Project** panel, navigate to Assets/Scenes/
2. You should see MainMenu.unity and Gameplay.unity
3. **If scenes are empty or missing the SceneBuilder:**
  - Open each scene
  - Create an empty GameObject ( GameObject → Create Empty )
  - Name it “**SceneBuilder**”
  - Attach the **RuntimeSceneBuilder** script to it
  - Save the scene ( Ctrl+S )
4. Repeat for both MainMenu and Gameplay scenes

## Step 4 - Configure Build Settings

1. Go to **File → Build Settings**
2. Add scenes in this exact order:
  - Assets/Scenes/MainMenu.unity (index 0)
  - Assets/Scenes/GamePlay.unity (index 1)
3. Drag scenes from the Project panel into the “Scenes In Build” list
4. Make sure **MainMenu** is at index **0** (it loads first)
5. Set **Target Platform** to **Windows**
6. Set **Architecture** to **x86\_64**

## Step 5 - Build the Game

1. In **Build Settings**, click “**Build**”
2. Choose an output folder (e.g., Build/ )
3. Wait for the build to complete
4. Your executable will be at: Build/Space Shooter.exe

## Step 6 - Run the Game

Double-click **Space Shooter.exe** to play!

## Game Controls

| Key                              | Action               |
|----------------------------------|----------------------|
| <b>Arrow Keys</b> or <b>WASD</b> | Move the player ship |
| <b>Space</b>                     | Shoot                |
| <b>Escape</b>                    | Pause / Resume       |

---

## Game Features

---

### Player

- 5 health points (displayed in HUD)
- Three weapon levels: single → double → spread shot
- Collectible shield that absorbs one hit
- Screen-boundary clamping

### Enemies

- **5 types:** Straight, Zigzag, Sine-wave, Charger, Boss
- Difficulty increases each wave (more enemies, faster, tougher)
- Boss enemies appear every 5th wave with spread-shot attacks
- Enemies can shoot (aimed, straight, and spread patterns)

### Power-Ups

- **Health** (green) – restores 2 HP
- **Weapon Upgrade** (orange) – increases weapon level for 10 seconds
- **Shield** (blue) – absorbs the next hit for 5 seconds
- Drop from defeated enemies (15% chance) + random spawns

### Scoring

- Base score per enemy type (100–250 points)
- **Wave multiplier:** +25% per wave (wave 5 = 2x score)
- Boss kills award 1000+ points
- High score saved persistently (PlayerPrefs)

### Visual

- Procedurally generated pixel-art sprites (no external assets needed)
- Dual-layer parallax scrolling star-field background
- Damage flash effects
- Color-coded bullets (green = player, red = enemy)
- Color-coded power-ups

### Audio

- Runs silently if no audio clips are provided
  - Full AudioManager ready for: shoot, explosion, hit, shield, power-up, button, wave-start SFX
  - Separate music tracks for menu and gameplay
-

## Project File Structure

```

space_shooter_game/
├── Assets/
│   ├── Scripts/
│   │   ├── PlayerController.cs
│   │   ├── EnemyController.cs
│   │   ├── BulletController.cs
│   │   ├── EnemySpawner.cs
│   │   ├── GameManager.cs
│   │   ├── PowerUpController.cs
│   │   ├── PowerUpSpawner.cs
│   │   ├── ParallaxBackground.cs
│   │   ├── AudioManager.cs
│   │   ├── MenuManager.cs
│   │   ├── UIManager.cs
│   │   ├── RuntimeSceneBuilder.cs
│   │   ├── SpriteGenerator.cs
│   │   ├── GameplaySceneBootstrap.cs
│   │   └── MainMenuBootstrap.cs
│   ├── Scenes/
│   │   ├── MainMenu.unity
│   │   └── Gameplay.unity
│   ├── Prefabs/
│   ├── Resources/
│   │   ├── Sprites/
│   │   └── Audio/
│   ├── ProjectSettings/
│   │   ├── ProjectSettings.asset
│   │   ├── TagManager.asset
│   │   ├── Physics2DSettings.asset
│   │   ├── InputManager.asset
│   │   ├── QualitySettings.asset
│   │   ├── EditorBuildSettings.asset
│   │   ├── AudioManager.asset
│   │   └── TimeManager.asset
│   ├── Packages/
│   │   ├── manifest.json
│   └── BUILD_INSTRUCTIONS.md
└── (Prefabs created at runtime)
    (Optional custom sprites)
    (Optional audio clips)
  
```

Player movement, shooting, health, shield  
 Enemy types, movement patterns, shooting  
 Bullet movement, lifetime, damage  
 Wave-based enemy spawning system  
 Central game state, score, scene management  
 Power-up types **and** player interaction  
 Random **and** drop-based power-up spawning  
 Scrolling star-field background  
 Sound effects **and** music management  
 Main menu, pause, game over screens  
 HUD (score, health, wave display)  
 Builds all game objects at runtime  
 Procedural pixel-art sprite creation  
 Initializes gameplay scene  
 Initializes main menu scene

## Architecture Overview

### How It Works (No Manual Scene Setup Needed)

The game uses a `RuntimeSceneBuilder` pattern:

1. Each scene ( `MainMenu` , `GamePlay` ) contains a single `SceneBuilder` `GameObject`
2. The `RuntimeSceneBuilder.cs` script detects which scene loaded
3. It **programmatically creates** all `GameObjects`, UI, prefabs, and wiring
4. This means you don't need to drag-and-drop anything in the Unity Inspector

### Singleton Managers

- **GameManager** – Persists across scenes ( `DontDestroyOnLoad` ), manages game state
- **AudioManager** – Persists across scenes, central SFX/music control
- **UIManager** – Per-scene, manages HUD elements

- **MenuManager** – Per-scene, manages menu panels
- **EnemySpawner** – Per-scene, handles wave-based spawning
- **PowerUpSpawner** – Per-scene, handles power-up generation

## Collision System

| Object A | Object B     | Result                    |
|----------|--------------|---------------------------|
| Player   | EnemyBullet  | Player takes 1 damage     |
| Player   | Enemy        | Player takes 2 damage     |
| Enemy    | PlayerBullet | Enemy takes bullet damage |
| Player   | PowerUp      | Power-up effect applied   |

## Troubleshooting

### “Script not found” errors

- Make sure all `.cs` files are in `Assets/Scripts/`
- Unity compiles them automatically; wait for compilation to finish

### Tags not recognized

- Manually add tags in **Edit → Project Settings → Tags and Layers**
- Required: `Player` , `PlayerBullet` , `EnemyBullet` , `Enemy` , `PowerUp`

### Scene is blank when playing

- Ensure each scene has a **SceneBuilder** GameObject with **RuntimeSceneBuilder** attached
- Check the Console for errors ( `Window → General → Console` )

### Build fails

- Verify scenes are in Build Settings in correct order (MainMenu first)
- Ensure Windows Build Support module is installed in Unity Hub
- Try **File → Build Settings → Switch Platform** to Windows first

### No sound

- This is normal! The game runs without audio files
- Add `.wav` / `.ogg` clips to `Assets/Resources/Audio/` and assign in Inspector

## Recommended Unity Versions

| Version               | Status                                             |
|-----------------------|----------------------------------------------------|
| Unity 2022.3 LTS      | ✓ <b>Recommended</b>                               |
| Unity 2021.3 LTS      | ✓ Fully compatible                                 |
| Unity 2023.x          | ✓ Should work (may need minor adjustments)         |
| Unity 6 (2024+)       | ⚠ May need UI updates (legacy Text → Text-MeshPro) |
| Unity 2020.x or older | ✗ Not recommended                                  |

## Optional Enhancements

Once the base game is working, you can easily add:

- **Particle effects** for explosions and thruster trails
- **Screen shake** on damage
- **Custom sprite art** (replace procedural sprites)
- **Sound effects** (See `Assets/Resources/Audio/README_AUDIO.txt` )
- **More enemy types** by extending the `EnemyType` enum
- **Difficulty settings** in the main menu
- **Leaderboard** using Unity's PlayerPrefs or a backend service

Built with Unity + C# | All code is complete and ready to compile